PROJECT

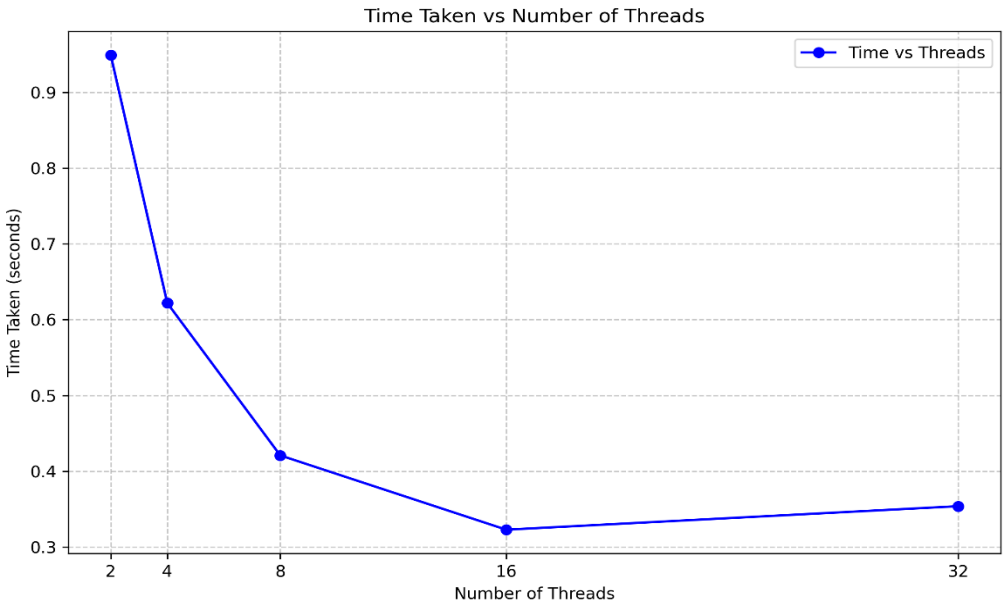
Parallelized K-Means Clustering for Efficient Data Analysis

Author -- Vaibhav Barsaiyan [cs24mtech12008]

Graphical Visualizations & Analysis

PARALLEL K-MEANS CLUSTERING ALGORITHM

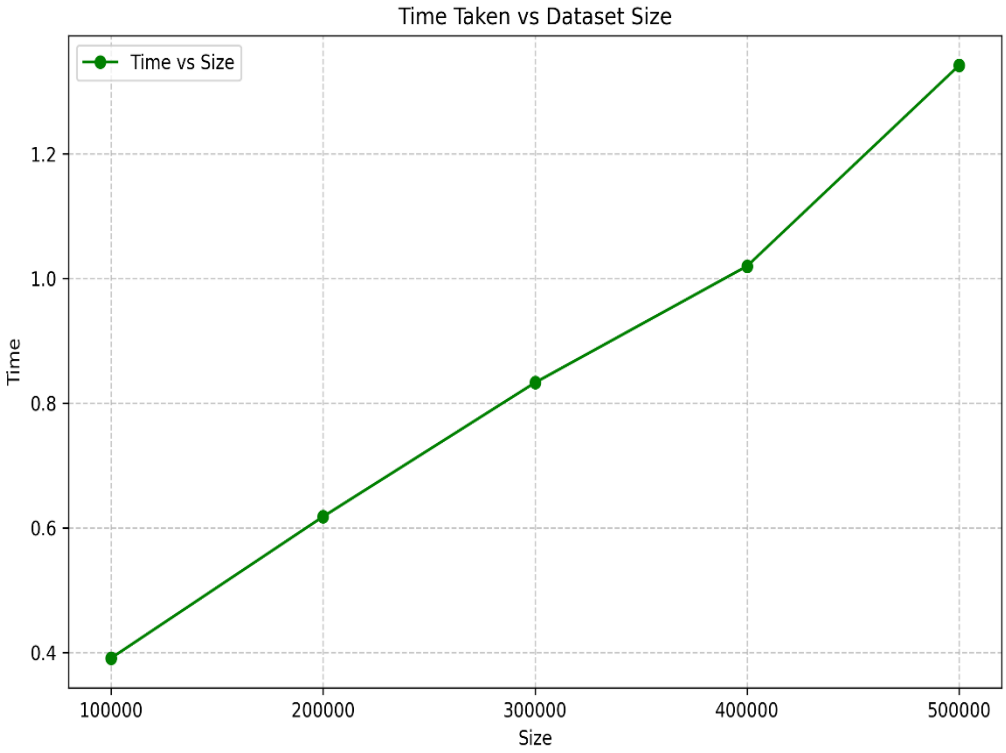
1] Varying the number of threads



- **Overall Trend:** As the number of threads increases, the time taken generally decreases, but not linearly.

- **Specific Observations:**
    - Using 1 thread takes the longest time.
    - Using 2-4 threads significantly reduces the time.
    - Increasing threads further provides diminishing returns.
    - Using 32 threads doesn't improve much over 16 threads.

2] Varying the data-set size



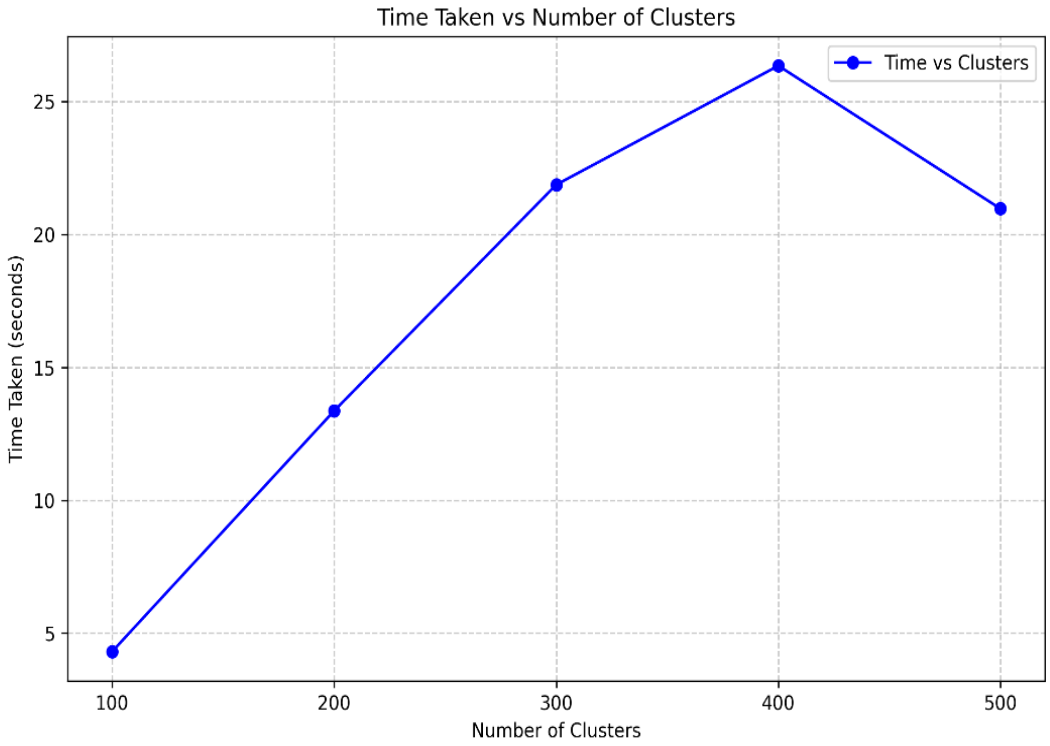
Overall Trend:

- As the dataset size increases, the time taken for the task also increases. This is a linear relationship, meaning the time taken is directly proportional to the dataset size.

Specific Observations:

- When the dataset size is 100,000, the time taken is around 0.4 units.
- As the size doubles to 200,000, the time roughly doubles to 0.6 units.
- This pattern continues for larger dataset sizes.

3] Varying the number of Clusters



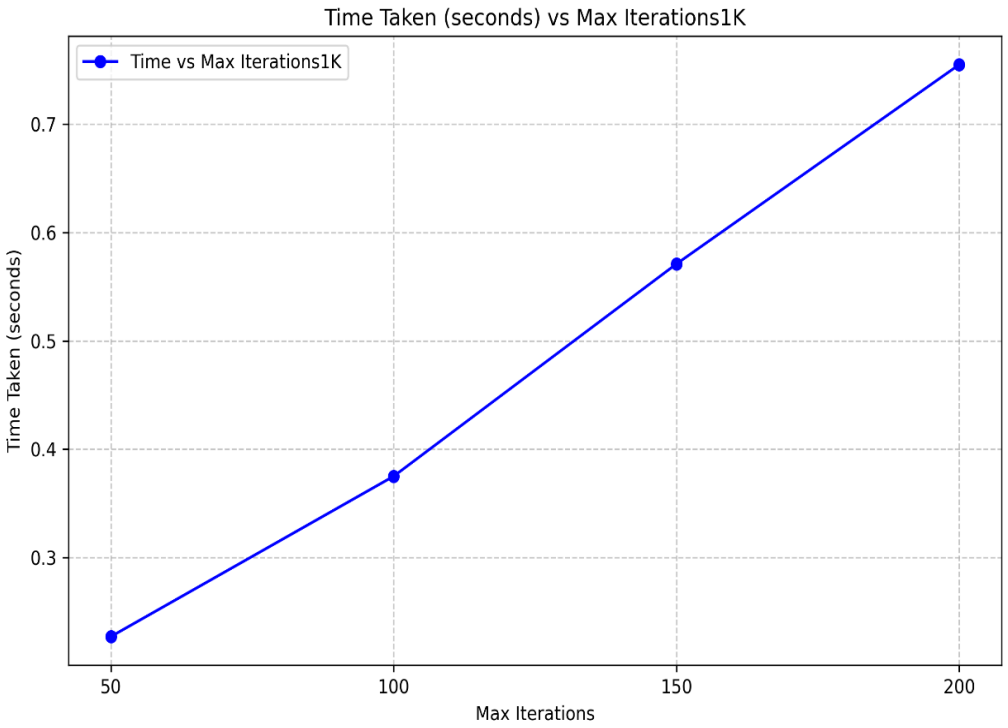
Overall Trend:

- As the number of clusters increases, the time taken initially increases, reaches a peak, and then decreases.

Specific Observations:

- When the number of clusters is 100, the time taken is around 5 seconds.
- As the number of clusters increases to 200, the time increases to around 13 seconds.
- The time continues to increase, reaching a peak of around 27 seconds at 400 clusters.
- After that, the time decreases to around 21 seconds at 500 clusters.

4] Varying the Maximum Allowed Number of Iterations



Overall Trend:

- As the maximum number of iterations increases, the time taken also increases linearly.

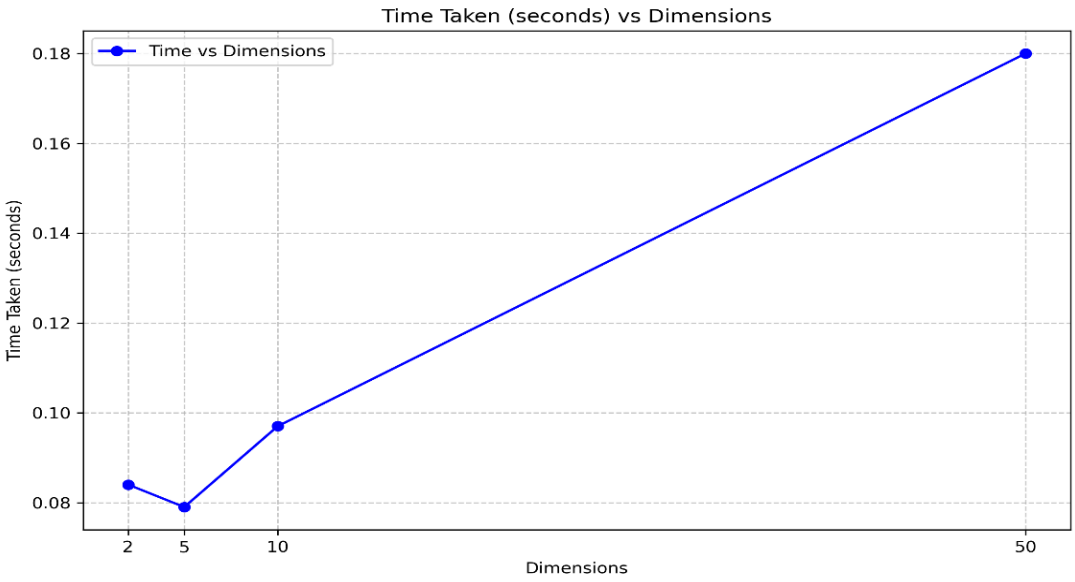
Specific Observations:

- When the maximum iterations are 50, the time taken is around 0.25 seconds.
- Doubling the iterations to 100 roughly doubles the time to around 0.5 seconds.
- This pattern continues for larger iteration values.

5] Varying the Initialization Method (Random vs. K-Means++)

Initialization Method	Time Taken (seconds)
K-Means++	0.211
Random	0.204

6] Varying the Dimensionality of Data (Feature Size)



Overall Trend:

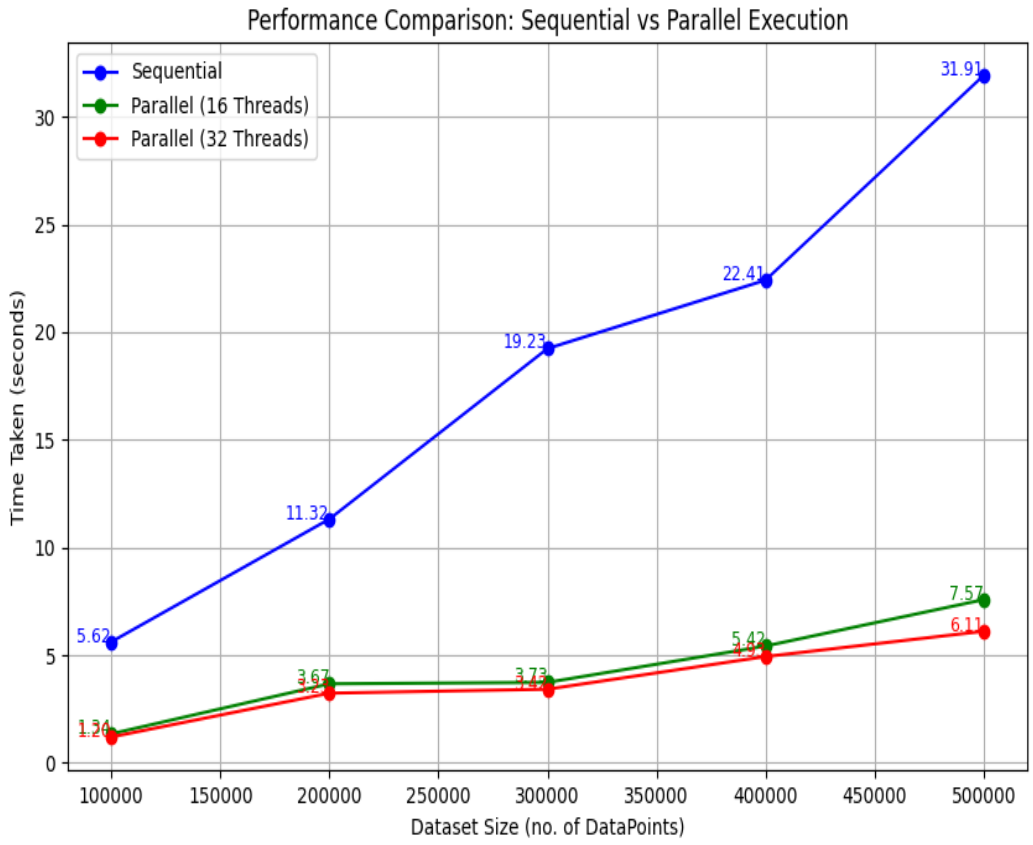
- As the number of dimensions increases, the time taken also increases.

Specific Observations:

- When the dimensions are 2, the time taken is around 0.08 seconds.
- Increasing the dimensions to 5 slightly increases the time to around 0.085 seconds.
- Further increasing the dimensions to 10 leads to a significant increase in time to around 0.1 seconds.
- The time continues to increase linearly with the number of dimensions, reaching around 0.18 seconds at 50 dimensions.

COMPARISION OF SEQUENTIAL & PARALLEL K-MEANS CLUSTERING ALGORITHM OBSERVATIONS

Case-1 : Varying the size of DataSet



[Fixed parametre : no. of clusters = 20 **]**

Overall Trend:

- As the dataset size increases, the time taken for all execution modes (sequential, parallel 16 threads, parallel 32 threads) also increases.
- Parallel execution generally outperforms sequential execution, especially for larger datasets.
- Increasing the number of threads in parallel execution can further reduce execution time, up to a certain point.

Specific Observations:

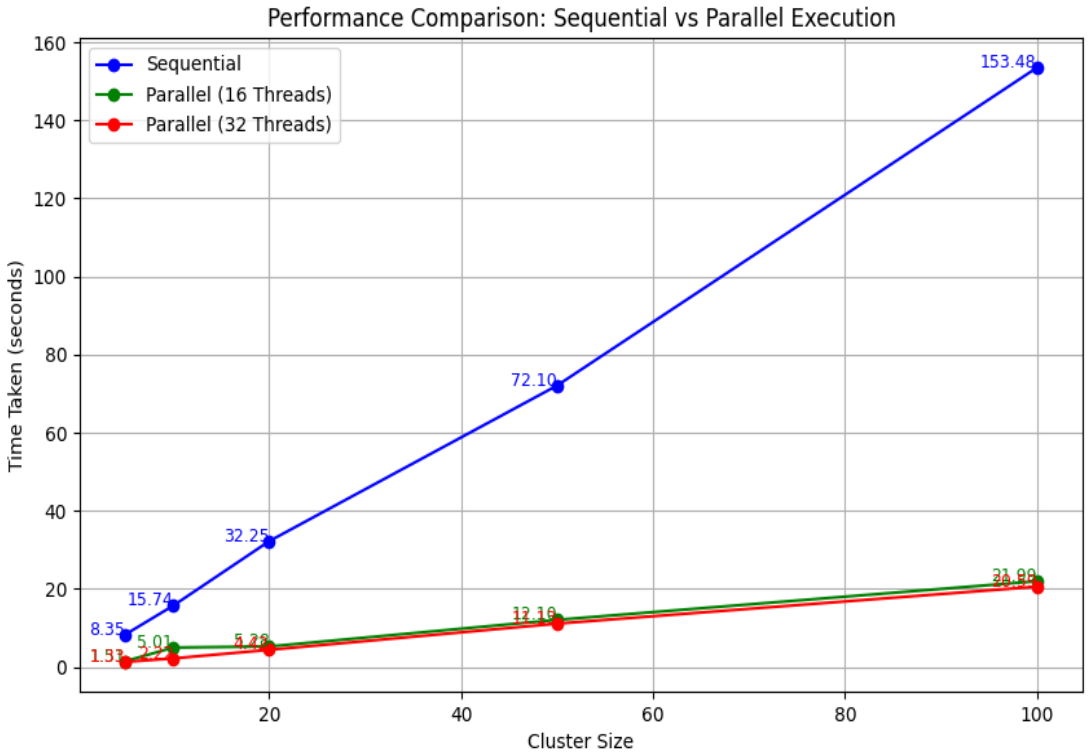
- As the dataset size increases, the advantage of parallel execution becomes more pronounced.
- Using 16 threads significantly reduces the time compared to sequential execution, especially for larger datasets.

- Using 32 threads can further reduce the time, but the improvement is less significant compared to using 16 threads.

Possible Explanations:

- **Overhead:** Adding more threads introduces overhead costs, such as thread creation, synchronization, and context switching. These costs can offset the benefits of parallelization for smaller datasets.
- **Task Granularity:** The task might not be easily divisible into small, independent subtasks, limiting the effectiveness of parallelization.
- **System Limitations:** The system's hardware and software resources might be limiting the performance of parallel execution, especially when using a large number of threads.

Case-2 : Varying the number of Clusters



[**Fixed parametres :** size of Dataset = 500000 datapoints]

Overall Trend:

- As the cluster size increases, the time taken for all execution modes (sequential, parallel 16 threads, parallel 32 threads) also increases.
- Parallel execution generally outperforms sequential execution, especially for larger cluster sizes.
- Increasing the number of threads in parallel execution can further reduce execution time, up to a certain point.

Specific Observations:

- For smaller cluster sizes (around 10), sequential execution is slightly faster than parallel execution. This is due to the overhead associated with creating and managing threads, which can outweigh the benefits of parallelism for small tasks.
- As the cluster size increases, the advantage of parallel execution becomes more pronounced.

- Using 16 threads significantly reduces the time compared to sequential execution, especially for larger cluster sizes.
- Using 32 threads can further reduce the time, but the improvement is less significant compared to using 16 threads.

Possible Explanations:

- **Task Granularity:** The task might be well-suited for parallelization, as it can be divided into smaller, independent subtasks that can be executed concurrently.
- **Communication Overhead:** As the cluster size increases, communication between threads might become a bottleneck, limiting the performance gains from parallel execution.
- **System Resources:** The system's hardware (CPU, memory) and software (operating system, libraries) can influence the performance of parallel execution, especially when using a large number of threads.
- **Synchronization:** If threads need to synchronize their work, it can introduce additional overhead, potentially limiting the performance gains.

LOW LEVEL DESIGN ANALYSIS (in brief)

SEQUENTIAL

K-MEANS CLUSTERING ALGORITHM IMPLEMENTATION

Low-Level Design for Sequential K-Means Code

- **Input and Data Setup**
    - Define Point structure with x, y coordinates and cluster (for cluster assignment).
    - Define Centroid structure with x, y coordinates.
    - Generate a dataset of points with random coordinates.
    - Initialize centroids using the K-Means++ algorithm.
- **Main Functional Modules**

1. **Cluster Assignment**
    - Iterate through all points.
    - For each point, calculate the distance to all centroids.
    - Assign the point to the nearest centroid's cluster.
2. **Centroid Update**
    - Iterate through all points to calculate the total x and y coordinates for each cluster.
    - Calculate the mean x and y coordinates for each cluster to update the centroid position.
3. **Silhouette Score Calculation**
    - For each point, calculate:
        - **Intra-cluster distance (a):** Average distance to points in the same cluster.
        - **Inter-cluster distance (b):** Minimum average distance to points in other clusters.
    - Compute the silhouette score for each point as (b - a) / max(a, b).
4. **Save Results**
    - Write silhouette scores to a CSV file with columns for point coordinates, cluster, and silhouette score.
    - Log the execution time for the sequential K-Means process in a CSV file.

- **Workflow in main Function**
    1. Initialize data points and centroids.
    2. Execute the K-Means algorithm iteratively:
        - Assign points to clusters.
        - Update centroids.
    3. Save silhouette scores and execution time to respective CSV files.
- **Output**
    - silhouette_scores.csv with the silhouette scores for each point.
    - Execution time logged in kmeans_time_results.csv.

This implementation retains the functionality of the original code but executes all operations sequentially, without threading or parallelism.

PARALLEL

K-MEANS CLUSTERING ALGORITHM IMPLEMENTATION

Low-Level Design for Parallel K-Means Code

- **Input and Data Setup**
    - Define Point structure to hold x, y coordinates and cluster assignments.
    - Define Centroid structure to hold x, y coordinates.
    - Generate random data points and initialize centroids using the K-Means++ method.
- **Main Functional Modules**

1. **Cluster Assignment (Parallel)**
    - **Function:** assignClusters
    - **Task:**
        - Divide the dataset into chunks based on the number of threads.
        - Each thread calculates the distance of assigned points to centroids and assigns them to the nearest cluster.
    - **Mechanism:** Each thread operates on a subset of the dataset.
2. **Centroid Update (Parallel)**
    - **Function:** updateCentroids
    - **Task:**
        - Each thread computes partial sums (x, y) and counts for its assigned points.
    - **Global Update:**
        - Use a mutex lock to combine thread-local updates into the global centroid data.
3. **K-Means Algorithm**
    - **Function:** kMeans
    - **Task:**
        - Iterate over a fixed number of iterations.
        - Perform cluster assignment and centroid updates using multithreading for both steps.
4. **Silhouette Score Calculation**

- **Function:** silhouetteScore
- **Task:**
  - Compute intra-cluster distances (a) and inter-cluster distances (b) for each point.
  - Calculate silhouette scores using the formula (b - a) / max(a, b).
- **Output:** Average silhouette score across all points.

5. **Data Output**
   - **Function:** saveSilhouetteScoresToCSV
   - **Task:** Write the silhouette score and cluster information for each point to a CSV file.
6. **Execution Time Logging**
   - **Output File:** kmeans_time_results.csv
   - **Task:** Record execution time for different thread counts (16, 32).

- **Workflow in main Function**
  1. Initialize random points and centroids.
  2. Execute the K-Means algorithm for increasing thread counts:
     - Measure and record execution time for each run.
  3. Save silhouette scores and performance metrics to CSV files.
  4. Destroy the mutex lock after completing all operations.
- **Output**
  - **Silhouette Scores:** silhouette_scores.csv with columns for point coordinates, cluster, and silhouette score.
  - **Execution Times:** kmeans_time_results.csv with columns for thread count and execution time.

This implementation uses pthreads for parallelism, dividing tasks like cluster assignment and centroid updates among multiple threads for faster execution.
